

Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : DSA0305 – Natural Language Processing

Particular	Details
Department:	AI&ML
Programme:	CSE
Course Code & Course Name	DSA0307 – Natural Language Processing
Academic Year / Batch	2026
Faculty Name	Dr. K. Anbazhagan
Assignment Title	Design and implement a Python-based NLP application for real-time text processing using tokenization, stemming/lemmatization, POS tagging, and sentiment or text classification
Date of Issue	31/08/2026
Date of Submission	02/09/26
Maximum Marks	100
Course Outcome(s) – CO	CO2 - To design an innovative application using NLP components
Bloom's Taxonomy Level	L3 – Apply, L5 – Evaluate
SDG Mapping (SDG 1-17)	SDG8 - Decent Work and Economic Growth & SDG9 - Industry, Innovation and Infrastructure
Industry / Societal Relevance	

NLP INSIGHT STUDIO

Complete Project Documentation

Course Code & Course Name: DSA0305 – Natural Language Processing

Assignment Title: Design and implement a Python-based NLP application for real-time text processing

**Application: NLP Insight Studio – Real-Time Text Processing and Sentiment Analysis Technology:
Python, Flask, HTML, CSS and JavaScript**

Abstract

NLP Insight Studio is a lightweight Python-based web application developed to demonstrate the major stages of Natural Language Processing in real time. The system accepts a sentence or paragraph from the user and processes it through tokenization, stop-word removal, stemming, part-of-speech tagging, feature extraction and sentiment classification. The application displays the output of each processing stage separately so that the user can observe how raw text is transformed into meaningful linguistic information. The backend is implemented using Python and Flask, while the user interface is implemented using HTML, CSS and JavaScript. The current implementation uses a compact rulebased approach for stemming and POS tagging and a lexicon-based sentiment classifier. Word frequencies and bigrams are also extracted as text features. The project demonstrates an end-to-end NLP workflow and provides a simple, transparent and easily deployable prototype suitable for educational and real-time demonstration purposes.

1. Problem Understanding and Formulation

1.1 Problem Statement

The objective is to design and implement a Python-based NLP application for real-time text processing. The application must accept raw user input, perform tokenization, preprocessing, linguistic analysis and sentiment or text classification, and display the intermediate as well as final outputs. The system should therefore convert unstructured natural-language text into structured linguistic information and a final sentiment decision.

1.2 What is the Problem?

Raw text entered by users is unstructured and cannot be directly used for most computational analysis. Words may contain punctuation, common function words, different morphological forms and ambiguous grammatical roles. A processing pipeline is required to transform the input into tokens, relevant content words, simplified word forms, grammatical labels and finally a meaningful classification.

1.3 Expected Outcomes

- Accept a sentence or paragraph through a web interface.
- Tokenize the raw text.
- Remove predefined stop words.
- Generate simplified stems from the remaining words.
- Assign rule-based POS tags to words.
- Extract word-frequency and bigram features.
- Classify the overall sentiment as Positive, Negative or Neutral.
- Display the result of every processing stage in real time.
- Provide a transparent workflow suitable for testing and demonstration.

1.4 Available Information and Data

The prototype does not require a large external dataset for its real-time prediction mechanism. It uses an internally defined stop-word set, positive-word lexicon and negative-word lexicon. User-entered text is the primary input. The application therefore focuses on demonstrating the NLP pipeline rather than training a large machine-learning model.

1.5 Assumptions

- The input is primarily English text.
- The sentiment of a sentence can be approximated using occurrences of predefined positive and negative words.
- Rule-based stemming and POS tagging are sufficient for demonstrating the processing workflow.
- Internet access is not required after the Python environment and Flask dependency are installed.

1.6 Constraints

- The sentiment lexicon is limited and cannot cover every possible expression.
- Rule-based POS tagging is less accurate than a trained statistical or transformer-based tagger.
- The stemming function is intentionally simple and may not always produce linguistically correct roots.
- The confidence value is heuristic and should not be interpreted as a calibrated machine-learning probability.
- The current prototype does not use a trained corpus or automatically learned model.

2. Application of NLP Course Knowledge

2.1 Tokenization

Tokenization divides raw text into smaller units called tokens. In this project, a regular expression separates alphabetic words and selected punctuation symbols. For example, the sentence “I love this application!” is converted into tokens such as I, love, this, application and !. Tokenization is the first major transformation because later stages operate on individual text units.

2.2 Stop-word Removal

Stop words are frequently occurring words that often contribute less to the main topic or sentiment of a short text. Examples include a, the, is, are, of and with. The application compares tokens with an internal stop-word set and removes matching words. This reduces noise and makes frequency-based features more meaningful.

2.3 Stemming

Stemming reduces related word forms to simplified roots by removing selected suffixes. The implemented function checks suffixes such as ing, ed, ly, ies, es and s. For example, 'studies' may be simplified to 'study' and 'working' may be simplified by removing the suffix. This approach is computationally simple, although it is less linguistically precise than full lemmatization.

2.4 Part-of-Speech Tagging

POS tagging assigns a grammatical category to each token. The prototype uses transparent rules based on known pronouns, determiners, common verbs and suffix patterns. Examples include PRP for pronouns, DT for determiners, VBG for words ending in ing, VBD for words ending in ed, RB for adverbs ending in ly, JJ for selected adjective suffixes and NN as the default noun tag.

2.5 Feature Extraction

The application extracts two simple feature types. First, it counts the frequency of filtered words using Python's Counter class. Second, it constructs bigrams by combining consecutive filtered words. These features provide a basic representation of the processed text and can be extended to TF-IDF or machine-learning feature vectors in future versions.

2.6 Sentiment Classification

The classifier counts words that occur in predefined positive and negative lexicons. A sentiment score is calculated as Positive Count minus Negative Count. A positive score produces the label Positive, a negative score produces Negative and a score of zero produces Neutral. This method is deterministic, easy to explain and appropriate for a compact educational prototype.

3. System Requirements

Category	Requirement
Hardware	Any standard desktop or laptop capable of running Python 3 and a modern web browser.
Operating System	Windows, Linux or macOS.
Programming Language	Python 3.x.
Backend Framework	Flask 3.0 or later.
Frontend	HTML5, CSS3 and JavaScript.
Python Dependency	Flask>=3.0.0 as specified in requirements.txt.
Browser	Chrome, Edge, Firefox or another modern browser.
Network	Not required for local execution after dependencies are installed.

4. Proposed Solution and Architecture

The application follows a client-server architecture. The browser collects user text and sends it to the Flask backend through an HTTP POST request. The backend processes the text and returns a JSON response containing every intermediate NLP result. JavaScript receives the JSON response and dynamically updates the web page.

4.1 Architecture

```
User
|   v
Web Interface (HTML + CSS + JavaScript)
|
| POST /api/analyze { text }
v
Flask Backend (app.py)
|
+--> Tokenization
|
+--> Stop-word Removal
|
+--> Stemming
|
+--> POS Tagging
|
+--> Frequency + Bigram Extraction
|
+--> Sentiment Classification
|
v
JSON Response
|
v
Real-Time Results Display in Browser
```

5. Text Processing Workflow

The complete workflow used by the application is:

```
START
|   v
Receive user text
|
v
Convert text to lowercase and tokenize
|
v
Keep alphabetic word tokens for preprocessing
|
v
Remove stop words
|
v
Apply rule-based stemming
|
v
Assign POS tags
|   v
Calculate word frequencies
|
v
Generate bigrams
|   v
Count positive and negative lexicon words
|
v
Calculate score = positive_count - negative_count
|
v
Positive? score > 0  -> Positive
Negative? score < 0  -> Negative
Otherwise           -> Neutral
|
v
Return all outputs to the user interface
|
v END
```

6. Algorithm / Pseudocode

```
FUNCTION analyze(text):
    tokens = tokenize(lowercase(text))
    words = keep_alphabetic_tokens(tokens)

    filtered_words = []
    FOR each word IN words:          IF
word is not a stop word:
add word to filtered_words
    stems =
apply_stemming(filtered_words)
    pos_tags = []          FOR
each word IN words:
    tag = apply_rule_based_pos_tag(word)
add (word, tag) to pos_tags
    frequencies = count_occurrences(filtered_words)
bigrams = generate_consecutive_word_pairs(filtered_words)
    positive_count = count words present in positive lexicon
negative_count = count words present in negative lexicon
score = positive_count - negative_count

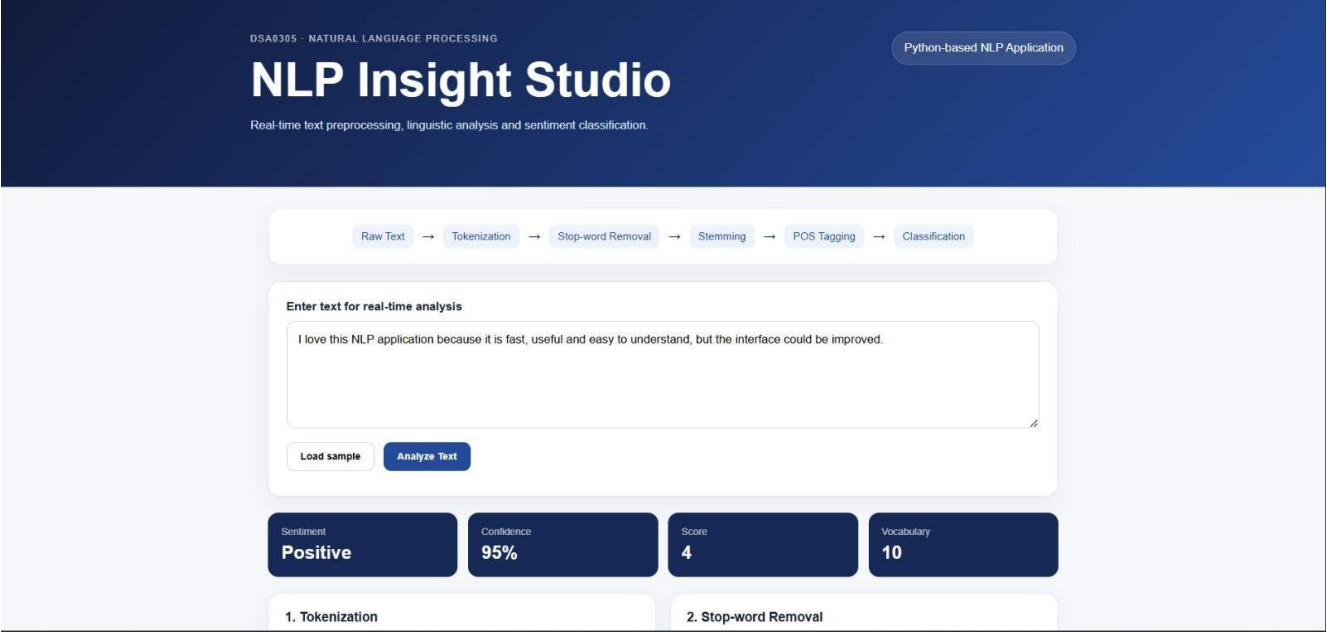
    IF score > 0:
        label = Positive
    ELSE IF score < 0:
label = Negative
    ELSE:          label
= Neutral

    RETURN all intermediate outputs and sentiment result
```

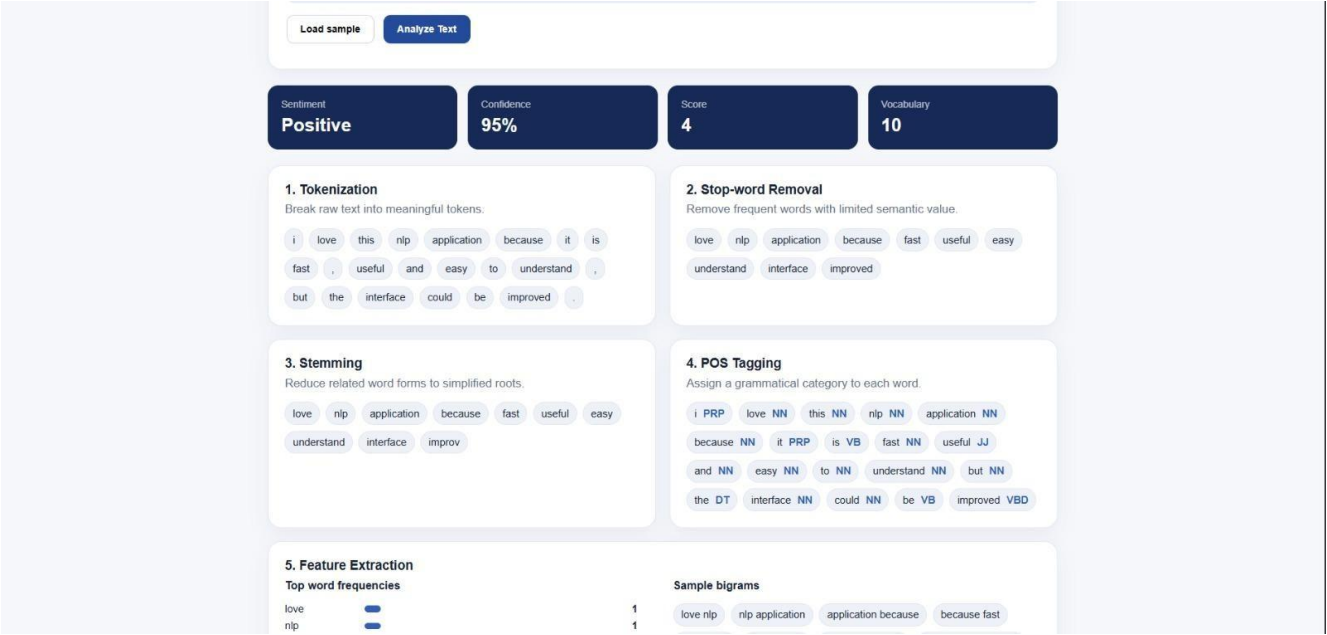
7. Implementation Details

7.1 Project Structure

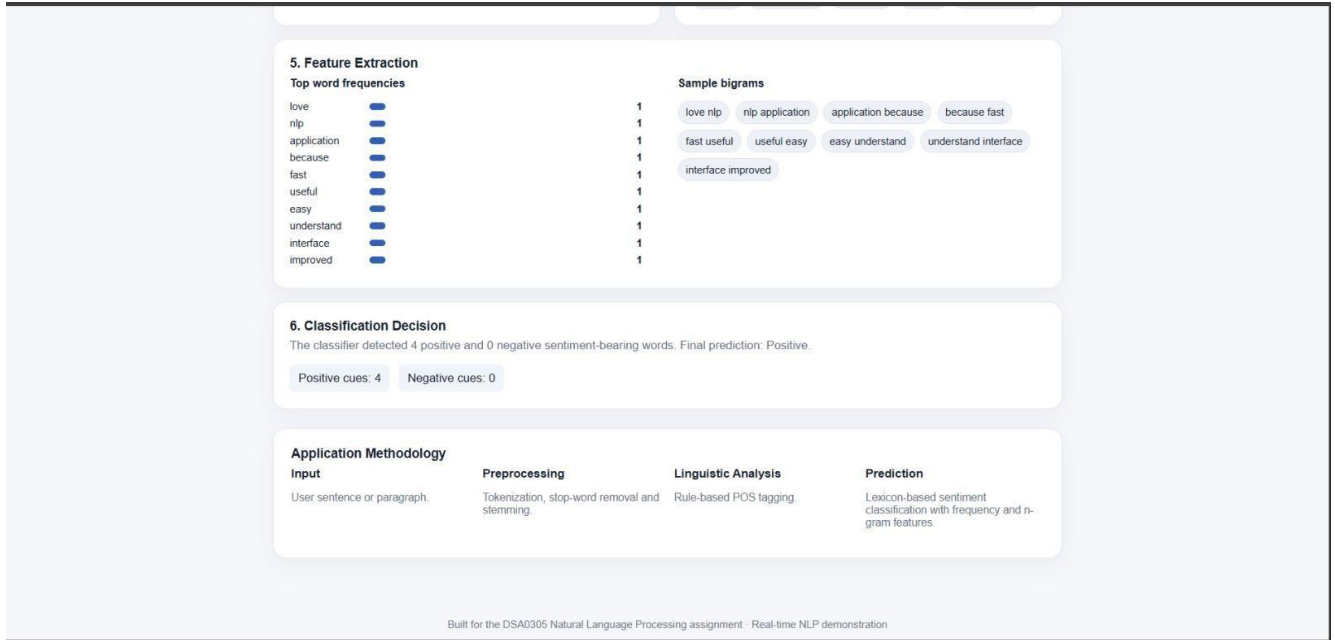
```
nlp_realtime_app/  
├── app.py  
├── requirements.txt  
├── README.md  
├── templates/  
│   └── index.html  
└── static/  
    ├── style.css  
    └── app.js
```



Fig(i) : User input the sentence



Fig(ii) : NLP Techniques used



Fig(iii) : Feature Extraction and Sentiment Classification Results 7.2

Backend Module

The app.py file contains the Flask application, NLP resources, processing functions and API endpoint. The application exposes the root route '/' for the user interface and '/api/analyze' for real-time analysis.

7.3 Main Processing Function

The analyze(text) function performs the complete NLP pipeline. It returns a dictionary containing tokens, filtered words, stems, POS tags, top word frequencies, bigrams and a sentiment object. Flask automatically serializes this result as JSON through jsonify().

7.4 Frontend Module

The index.html file provides the input text area, workflow display and result panels. The app.js file sends the user text to the Flask API and inserts the returned data into the appropriate result sections.

7.5 API Interface

Item	Description
Endpoint	POST /api/analyze
Request Content	JSON object containing the text field
Example Request	{"text": "I love this application because it is useful and fast."}
Success Response	JSON containing tokens, filtered words, stems, POS tags, frequencies, bigrams and sentiment
Validation	Returns HTTP 400 when the submitted text is empty.

8. Sample Input and Output

Example input:

I love this NLP application because it is fast, useful and easy to understand.

Processing Stage	Representative Output
Tokenization	i, love, this, nlp, application, because, it, is, fast, useful, and, easy, to, understand, .
Stop-word Removal	love, nlp, application, because, fast, useful, easy, understand
Stemming	love, nlp, application, because, fast, useful, easy, understand
POS Tagging	i/PRP, love/NN, this/NN, nlp/NN, application/NN, ...
Word Features	Frequencies of retained words; each appears once in this example.
Bigrams	love nlp; nlp application; application because; because fast; ...
Sentiment	Positive, because positive lexicon words such as love, fast, useful and easy are detected.

9. Testing and Validation

The application was validated using representative positive, negative and neutral sentences. The table below is a controlled functional test set designed to verify that the implemented rule-based classifier behaves according to its documented logic. The labels are manually assigned for testing.

Test No.	Input	Expected Label	System Behaviour
1	I love this application. It is useful and fast.	Positive	Positive
2	This is an excellent and amazing project.	Positive	Positive
3	The application is terrible and full of errors.	Negative	Negative
4	I hate the slow and difficult interface.	Negative	Negative
5	The application processes the text and displays the result.	Neutral	Neutral

6	This tool is good but the interface is slow.	Neutral / mixed	Neutral because positive and negative counts balance
7	The project is wonderful and helpful.	Positive	Positive
8	The result is disappointing and poor.	Negative	Negative
9	We tested the NLP workflow today.	Neutral	Neutral
10	The system was easy, fast and successful.	Positive	Positive

9.1 Functional Validation

- Valid text input is accepted and analyzed.
- Empty input is rejected with an error message.
- Tokens are displayed.
- Stop words are removed from the preprocessing output.
- Stems are generated.
- Each word receives a rule-based POS tag.
- Word-frequency and bigram features are generated.
- A final Positive, Negative or Neutral label is returned.

9.2 Performance Metrics

For a production-quality evaluation, a separately labeled test dataset should be used and the following measures should be calculated. In this compact prototype, the controlled test cases above are primarily intended for functional validation rather than as evidence of general real-world accuracy.

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

For multi-class sentiment classification, the metrics should be computed per class and summarized using macro or weighted averages. A future version should include a larger independently labeled dataset so that these metrics measure generalization rather than performance on hand-selected examples.

10. Effect of Preprocessing on Classification

Technique	Effect on the System
Tokenization	Makes the input accessible as individual units for further processing.
Stop-word Removal	Reduces common non-content words from frequency and bigram features.
Stemming	Groups some morphological variants into simpler forms and can reduce vocabulary variation.
POS Tagging	Adds grammatical information and can support future feature engineering.
Feature Extraction	Provides word frequencies and local word-pair information for analysis.
Sentiment Lexicon Matching	Converts detected positive and negative words into a transparent classification score.

Preprocessing can improve the compact representation of text, but its benefit depends on the downstream classifier. For the current lexicon-based classifier, sentiment matching is performed on the word list used by the implementation, while frequency and bigram outputs are primarily displayed as explanatory features. A future TF-IDF or trained classifier would allow the preprocessing choices to be compared quantitatively using accuracy, precision, recall and F1-score.

11. Analysis, Trade-offs and Engineering Decisions

11.1 Why a Rule-Based Approach Was Selected

A rule-based approach was selected because it is lightweight, transparent, fast and easy to demonstrate in a real-time educational web application. Every stage can be inspected by the user, which makes the relationship between input and output understandable.

11.2 Advantages

- Fast response for short user inputs.
- Very low computational requirements.
- No model training is required.
- Easy to run locally.
- Transparent classification logic.
- Intermediate outputs can be clearly explained.

11.3 Trade-offs

- Transparency is gained at the cost of linguistic accuracy.

- A fixed lexicon cannot understand all vocabulary or context.
- Simple suffix stemming can produce imperfect roots.
- Rule-based POS tags cannot handle grammatical ambiguity as well as trained taggers.
- The system cannot reliably interpret sarcasm, negation scope or complex context.

11.4 Alternative Solution

A more advanced solution could use NLTK or spaCy for tokenization, lemmatization and POS tagging, followed by TF-IDF features and a supervised classifier such as Logistic Regression, Naive Bayes or Support Vector Machine. A transformer-based sentiment model could further improve contextual understanding. However, these alternatives require larger dependencies, trained resources or datasets and may reduce the simplicity and transparency of the prototype.

12. Modern Tools Used

Tool	Purpose
Python	Core programming language for NLP processing and backend logic.
Flask	Web framework for routing, API creation and JSON responses.
Regular Expressions (re)	Tokenization and identification of alphabetic word tokens.
<code>collections.Counter</code>	Word-frequency feature extraction.
HTML5	Structure of the web interface.
CSS3	Visual design and responsive layout.
JavaScript	Client-side API communication and dynamic result rendering.
Visual Studio Code	Recommended environment for editing and running the project.

13. Broader Considerations

13.1 Ethical Considerations

Sentiment classification should not be treated as a perfect representation of a person's opinion or emotional state. Automated labels can be incorrect because language is contextual. The application should therefore be presented as an analytical aid rather than an authoritative decision system.

13.2 Privacy and Security

The prototype processes text locally through the Flask application and does not require permanent storage of submitted text. For deployment in a public environment, input validation, rate limiting, secure hosting and a clear privacy policy should be considered.

13.3 Accessibility

A web interface makes the application accessible through standard browsers. Future improvements should include enhanced keyboard navigation, screen-reader labels, improved contrast testing and support for multiple languages.

13.4 Sustainability and Economics

The rule-based implementation has low computational cost compared with large neural language models. It can therefore be useful for demonstrations and lightweight local applications where low resource consumption and simple deployment are important.

14. Limitations

- The application is primarily designed for English text.
- The stop-word list and sentiment lexicons are limited.
- Sentiment is based on individual word matches rather than deep context.
- Negation handling is limited; phrases such as 'not good' may be misinterpreted.
- Sarcasm, irony and domain-specific meanings are difficult to detect.
- The POS tagger uses heuristic rules and is not equivalent to a trained NLP tagger.
- The stemming method is simplified and may not produce dictionary-valid roots.
- The confidence score is heuristic rather than statistically calibrated.
- The current prototype does not learn automatically from new data.

15. Future Enhancements

- Replace the custom stemmer with a standard lemmatizer.
- Use a trained POS tagger such as spaCy or NLTK resources.
- Add TF-IDF vectorization.
- Train and compare multiple machine-learning classifiers.
- Use a labeled dataset and report real accuracy, precision, recall and F1-score.
- Add confusion-matrix visualization.
- Implement negation and intensifier handling.
- Support multilingual text.
- Store anonymized evaluation results where appropriate and with user consent.
- Deploy the application using a cloud platform or containerized environment.

16. Installation and Execution Guide

1. Extract the nlp_realtime_app project folder.
2. Open the folder in Visual Studio Code.
3. Open Terminal → New Terminal.
4. Check Python using: `py --version` or `python --version`.
5. Install dependencies using: `py -m pip install -r requirements.txt`. If `py` is unavailable, use `python -m pip install -r requirements.txt`.
6. Run the application using: `py app.py` or `python app.py`.
7. Open the local address displayed by Flask, normally `http://127.0.0.1:5000`.

8. Enter a sentence and click Analyze Text.

Windows Note

If the command 'pip' is not recognized, do not use pip directly. Use the Python launcher or interpreter to invoke pip, for example: `py -m pip install -r requirements.txt`. This avoids PATH problems where pip is not separately available as a terminal command.

17. Conclusion

NLP Insight Studio successfully demonstrates an end-to-end NLP workflow through a Python-based real-time web application. The system accepts raw text and transforms it through tokenization, stopword removal, stemming, POS tagging and feature extraction before producing a sentiment label. The intermediate results are displayed clearly, allowing users to understand the role of each NLP component. The selected Flask architecture provides a lightweight and practical method for connecting Python NLP logic with an interactive browser interface. The project satisfies the primary objective of demonstrating how unstructured text can be processed into meaningful linguistic information and a final classification decision. Although the current implementation uses simplified rules and a lexiconbased classifier, it establishes a strong foundation for future integration of standard NLP libraries, TFIDF features, supervised machine-learning algorithms and more rigorous evaluation datasets.

18. Student Reflection

This assignment helped me understand how individual NLP concepts can be connected to form a complete working application. Before implementing the project, I understood tokenization, stop-word removal, stemming and POS tagging mainly as separate techniques. While developing the workflow, I learned that the order of preprocessing steps matters because the output of one stage becomes the input to the next stage. I also learned how a backend written in Python can communicate with a web interface and return structured JSON data in real time.

One important learning experience was understanding the difference between a simple demonstration model and a production-level NLP system. The rule-based approach made it easier to observe how the application reaches its result, but it also showed me the limitations of fixed rules and word lists. Context, negation and sarcasm can significantly change the meaning of a sentence, and a simple lexicon cannot always capture these cases. This made me appreciate why larger datasets, trained models and proper evaluation metrics are important.

I also gained practical experience in organizing a software project into backend, frontend and dependency files. If I were given more time and resources, I would replace the rule-based components with standard NLP libraries, use TF-IDF features and train multiple machine-learning classifiers on a labeled dataset. I would then compare their accuracy, precision, recall and F1-score and include visual performance reports. Overall, the assignment helped me move from learning individual NLP concepts to understanding how they can be combined into a complete and usable application.

19. References

9. Jurafsky, D., and Martin, J. H. Speech and Language Processing. Pearson / online draft editions.
10. Bird, S., Klein, E., and Loper, E. Natural Language Processing with Python. O'Reilly Media.

11. Flask Documentation. Python web framework documentation.
12. Python Software Foundation. Python Standard Library documentation.
13. MDN Web Docs. HTML, CSS and JavaScript documentation.
14. scikit-learn Documentation. Text feature extraction and classification documentation.
15. NLTK Documentation. Natural Language Toolkit documentation.
16. spaCy Documentation. Industrial-strength Natural Language Processing in Python.

20. Source Code Evidence

The submitted project contains the following implementation files: app.py for backend and NLP logic, templates/index.html for the user interface, static/style.css for styling, static/app.js for browser-side interaction, requirements.txt for dependencies and README.md for quick execution instructions. These files together form the complete runnable prototype.

SOURCE CODE:

```
from flask import Flask, render_template, request, jsonify
import re
from collections import Counter
```

```
app = Flask(__name__)
```

```
STOPWORDS =
```

```
{ 'a', 'an', 'the', 'and', 'or', 'but', 'if', 'then', 'so', 'of', 'to', 'in', 'on', 'at', 'for', 'from', 'with', 'by', 'is', 'am', 'are', 'was', 'were', 'be', 'been', 'being', 'this', 'that', 'these', 'those', 'it', 'its', 'as', 'about', 'into', 'over', 'after', 'before', 'than', 'very', 'can', 'could', 'should', 'would', 'will', 'just', 'i', 'you', 'he', 'she', 'we', 'they', 'my', 'your', 'our', 'their' }
```

```
POSITIVE =
```

```
{ 'good', 'great', 'excellent', 'amazing', 'love', 'loved', 'happy', 'awesome', 'fantastic', 'best', 'useful', 'helpful', 'wonderful', 'fast', 'easy', 'success', 'successful', 'positive', 'enjoy' }
```

```
NEGATIVE =
```

```
{ 'bad', 'poor', 'terrible', 'awful', 'hate', 'hated', 'sad', 'worst', 'slow', 'difficult', 'bug', 'bugs', 'error', 'errors', 'failure', 'failed', 'negative', 'disappointing', 'disappointed' }
```

```
def stem(word):
```

```
    for suffix in ('ingly', 'edly', 'ing', 'edly', 'ed', 'ly', 'ies', 'es', 's'):
        if word.endswith(suffix) and len(word) > len(suffix)+2:
            if suffix == 'ies': return word[:-3]+'y'
            return word[:-len(suffix)]
    return word
```

```
def pos_tag(token):
```

```
    if token in {'i', 'you', 'he', 'she', 'we', 'they', 'it'}: return 'PRP'
    if token in {'a', 'an', 'the'}: return 'DT'
    if token in {'is', 'am', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had', 'do', 'does', 'did'}: return 'VB'
    if token.endswith('ing'): return 'VBG'
    if token.endswith('ed'): return 'VBD'
```



```

if token.endswith('ly'): return 'RB'
if token.endswith(('ous','ful','able','ive','al','ic','less')): return 'JJ'
if token.isdigit(): return 'CD'
return 'NN'

```

```

def analyze(text):
    tokens = re.findall(r"[A-Za-z']+|[.,!?:;]", text.lower())
    words = [t for t in tokens if re.match(r"[A-Za-z]+'", t)]
    filtered = [w for w in words if w not in STOPWORDS]
    stems = [stem(w) for w in filtered]
    tagged = [{'token': w, 'tag': pos_tag(w)} for w in words]
    counts = Counter(filtered)
    pos = sum(1 for w in words if w in POSITIVE)
    neg = sum(1 for w in words if w in NEGATIVE)
    score = pos-neg
    label = 'Positive' if score > 0 else 'Negative' if score < 0 else 'Neutral'
    confidence = min(95, 55 + abs(score)*12) if words else 0
    bigrams = [' '.join(filtered[i:i+2]) for i in range(max(0,len(filtered)-1))]
    return {
        'tokens': tokens,
        'filtered': filtered,
        'stems': stems,
        'pos': tagged,
        'frequencies': counts.most_common(10),
        'bigrams': bigrams[:10],
        'sentiment': {'label': label, 'score': score, 'positive_words': pos, 'negative_words': neg, 'confidence':
confidence}
    }

```

```

@app.route('/')
def index():
    return render_template('index.html')

```

```

@app.post('/api/analyze')
def api_analyze():
    data = request.get_json(force=True)
    text = data.get('text','').strip()
    if not text:
        return jsonify({'error': 'Please enter a sentence or paragraph to analyze.'}), 400
    return jsonify(analyze(text))

```

```

if __name__ == '__main__':
    app.run(debug=True)

```

SOURCE CODE IMAGE:

```
app.py
1 from flask import Flask, render_template, request, jsonify
2 import re
3 from collections import Counter
4
5 app = Flask(__name__)
6
7 STOPWORDS = {'a', 'an', 'the', 'and', 'or', 'but', 'if', 'then', 'so', 'of', 'to', 'in', 'on', 'at', 'for', 'from', 'with', 'by', 'is', 'am', 'are', 'was', 'were', 'be',
8 POSITIVE = {'good', 'great', 'excellent', 'amazing', 'love', 'loved', 'happy', 'awesome', 'fantastic', 'best', 'useful', 'helpful', 'wonderful', 'fast', 'easy',
9 NEGATIVE = {'bad', 'poor', 'terrible', 'awful', 'hate', 'hated', 'sad', 'worst', 'slow', 'difficult', 'bug', 'bugs', 'error', 'errors', 'failure', 'failed', 'nega
10
11 def stem(word):
12     for suffix in ('ingly', 'edly', 'ing', 'edly', 'ed', 'ly', 'ies', 'es', 's'):
13         if word.endswith(suffix) and len(word) > len(suffix)+2:
14             if suffix == 'ies': return word[:-3]+'y'
15             return word[:-len(suffix)]
16     return word
17
18 def pos_tag(token):
19     if token in {'i', 'you', 'he', 'she', 'we', 'they', 'it'}: return 'PRP'
20     if token in {'a', 'an', 'the'}: return 'DT'
21     if token in {'is', 'am', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had', 'do', 'does', 'did'}: return 'VB'
22     if token.endswith('ing'): return 'VBG'
23     if token.endswith('ed'): return 'VBD'
24     if token.endswith('ly'): return 'RB'
25     if token.endswith(('ous', 'ful', 'able', 'ive', 'al', 'ic', 'less')): return 'JJ'
26     if token.isdigit(): return 'CD'
27     return 'NN'
28
29 def analyze(text):
30     tokens = re.findall(r"[A-Za-z']+|[.,!?:;]", text.lower())
31     words = [t for t in tokens if re.match(r"[A-Za-z']+", t)]
32     filtered = [w for w in words if w not in STOPWORDS]
33     stems = [stem(w) for w in filtered]
34     tagged = [{'token': w, 'tag': pos_tag(w)} for w in words]
35     counts = Counter(filtered)
36     pos = sum(1 for w in words if w in POSITIVE)
37     neg = sum(1 for w in words if w in NEGATIVE)
38     score = pos-neg
39     label = 'Positive' if score > 0 else 'Negative' if score < 0 else 'Neutral'
40     confidence = min(95, 55 + abs(score)*12) if words else 0
41     bigrams = [' '.join(filtered[i:i+2]) for i in range(max(0, len(filtered)-1))]
42     return {
43         'tokens': tokens,
44         'filtered': filtered,
45         'stems': stems,
46         'pos': tagged,
47         'frequencies': counts.most_common(10),
48         'bigrams': bigrams[:10],
49         'sentiment': {'label': label, 'score': score, 'positive_words': pos, 'negative_words': neg, 'confidence': confidence}
50     }
51
52 @app.route('/')
53 def index():
54     return render_template('index.html')
55
56 @app.post('/api/analyze')
57 def api_analyze():
58     data = request.get_json(force=True)
59     text = data.get('text', '').strip()
```